

An
Industrial Training Report

on
C++ Programming

Submitted in partial fulfillment for the award of degree of
BACHELOR OF TECHNOLOGY

In
Computer Science & Engineering



Submitted to:

Dr. Pradeep Jha

HOD-CSE/IT/AI&DS/IOT/CYBER SECURITY Dept

Submitted By:

Himanshu Jain

24EGJCS095

**Department of Computer Science &
Engineering GLOBAL INSTITUTE OF
TECHNOLOGY JAIPUR
(RAJASTHAN)-302022
SESSION: 2025-26**

Certificate



Acknowledgement

I take this opportunity to express my deep sense of gratitude to my coordinator **Mr. AshaRam Gurjar**, Assistant Professor Department of Computer Science and Engineering, Global Institute of Technology, Jaipur, for his valuable guidance and cooperation throughout the Practical Training work. He provided constant encouragement and unceasing enthusiasm at every stage of the Practical Training work.

We are grateful to our respected **Dr. I. C. Sharma, Principal GIT** for guiding us during Practical Training period

We express our indebtedness to **Dr. Pradeep Jha**, Head of Department of Computer Science and Engineering, Global Institute of Technology, Jaipur for providing me ample support during my Practical Training period.

Without their support and timely guidance, the completion of our Practical Training would have seemed a farfetched dream. In this respect we find ourselves lucky to have mentors of such a great potential.

Place: GIT, Jaipur

Himanshu Jain

24EGJCS095

B.Tech. III Semester, II Year, CSE

Abstract

This is Specially Designed course to covers C++ from very basic to Ultimate Level. This course covers C++ from very basic to more advanced features. Maybe you have some experience with other programming languages, but want to learn C. It's a great language to add to your resume!. The object oriented programming concepts are clearly explained, we will learn classes, objects, inheritance, polymorphism, Operator overloading, Data Structure, Pointer, file handling, Dynamic Memory allocation, Recursion, apart from basic programming concepts like variables, branching and looping, functions, reference parameters, arrays, string, vectors hands on the real life project in C++.

C++ allows we to have a lot of control as to how we use computer resources, so in the right hands its speed and ability to cheaply use resources should be able to surpass other languages. Thanks to C++'s performance, it is often used to develop game engines, games, and desktop apps.

After completing this course we will be confident enough to take up any challenging problem in coding using C++ Programming.

Table of Contents

Certificate.....	ii
Acknowledgement.....	iii
Abstract.....	iv
Table of Content.....	v
List of Tables.....	viii
List of Figures.....	ix
Chapter: 1 Introduction & Overview.....	11
1.1 The design & Evolution of C++.....	11
1.2 C++ Design Aims.....	11
1.3 C Programming Model.....	12
Chapter: 2 Storage.....	13
2.1 Compile, Link & Execute.....	14
2.2 Type Casting.....	14
2.3 Concrete Types.....	15
Chapter: 3 C++ Standard Library.....	16
3.1 Strings & I/O.....	17
3.2 Vector Arithmetic.....	17
Chapter: 4 Object Oriented Programming.....	19
4.1 Basic Concept of OOPs.....	19
4.1.1 Objects.....	19
4.1.2 Classes.....	19
4.1.3 Data Abstraction.....	20
4.1.4 Data Encapsulation.....	20
4.1.5 Inheritance.....	20
4.1.6 Polymorphism.....	20

Chapter: 5 Function in C++.....	23
5.1 The main() Function.....	23
5.2 Inline Function.....	23
5.3 Default Argument.....	25
Chapter: 6 Class.....	27
6.1 Defining member Function.....	28
6.2 Outside the class Definition.....	28
6.3 A C++ Program with Class.....	29
6.4 Private Member Function.....	31
Chapter: 7 Constructors.....	33
7.1 Example of Default Constructor.....	34
7.2 Parameterized Constructor.....	35
7.3 Copy Constructor.....	36
Chapter: 8 Type Constructor.....	37
8.1 Basic to Class Type.....	37
8.2 Class to Basic Type.....	39
8.3 One Class to another Class Type.....	40
Chapter: 9 Inheritance.....	44
9.1 Defining Derived Classes.....	44
9.2 Single Inheritance.....	47
9.3 Making a Private Member Inheritance.....	47
9.3.1 Private.....	47
9.3.2 Public.....	47
9.3.3 Protected.....	47
9.4 Multilevel Inheritance.....	50
9.5 Hierarchical Inheritance.....	50

Chapter: 10 Polymorphism.....	51
10.1 Introdction	51
10.2 Pointers to Objects.....	51
10.3 this Pointer.....	52
Chapter: 11 Exception Handling.....	53
11.1 Exception Handling Model.....	53
11.2 Throw Construct.....	55
11.3 Catch Construct.....	55
11.4 Try Construct	56
Chapter: 12 Managing Console I/O.....	57
12.1 Introduction.....	57
12.2 The put() & get() Function.....	58
12.3 The getline() & write() Function.....	59
12.4 Formatted Console I/O Operations.....	60
12.5 Streams.....	61
12.6 I/O Operations.....	62
12.7 Put() & get() function.....	62
12.8 Keywords.....	63
Chapter: 13 Namespaces.....	64
13.1 Scopes.....	64
13.2 Namespaces.....	64
• Conclusion	
• References	

List of Tables

- **Table 9.1:** Inheritance Visibility38
- **Table 12.1:** I/O Manipulators50
- **Table 12.2:** I/O Manipulators and Their Equivalent Ios Function.....51

List of Figures

• Figure 9.1: Multilevel Inheritance.....	38
• Figure 9.2: Multiple Inheritance.....	39
• Figure 9.3: Hierarchical Inheritance.....	40
• Figure 9.4: Hierarchical Inheritance Approach.....	40
• Figure 10.1: Memory Representation of A Variable.....	41
• Figure 11.1: Exception Handling Flowchart.....	45

List of Symbols

- **BLAS**- Basic Linear Algebra Subprograms
- **I/O**- Input-output
- **OOP**- Object Oriented Programming
- **IOS**- Input-Output Stream
- **CIM**- Computer Integrated Manufacturing
- **CAD**- Computer Aided Design
- **CAM**- Computer Aided Manufacturing

INTRODUCTION AND OVERVIEW

C++ programming language provides a model of memory and computation that closely that of most computers. In additions, it provides powerful and flexible mechanisms for abstraction; that is, languages constructs that allow the programmer to introduce and use new type of objects that match the concepts of an application. Thus, C++ supports styles of programming that rely on fairly direct manipulation of hardware resources to deliver a high degree of efficiency plus higher-level styles of programming that rely on user-defined types by a computer.

1.1 The design and Evolution of C++

C++ was designed and implemented by Bjarne Stroustrup (the author of this article) at AT&T Bell Laboratories to combine the organizational and design strengths of Simula with C's facilities for systems programming. The initial version of C++, called "C with Classes" [Stroustrup,1980], was first used in 1980; it supported traditional system programming techniques (§3) and data abstraction (§4.1). The basic facilities for object-oriented programming (§4.2-4.3) were added in 1983 and object-oriented design and programming techniques were gradually introduced into the C++ community. The language was first made commercially available in 1985 [Stroustrup,1986] [Stroustrup,1986b]. Facilities for generic programming.

1.2 C++ Design Aims

C++ was designed to deliver the flexibility and efficiency of C for systems programming together with Simula's facilities for program organization (usually referred to as object-oriented programming). Great care was taken that the higher-level programming techniques from Simula could be applied to the systems programming domain. That is, the abstraction mechanisms provided by C++ were specifically designed to be applicable to programming tasks that demanded the highest degree of efficiency and flexibility.

1.3 The C Programming Model

A fundamental property of computers in widespread use has remained remarkably constant: Memory is a sequence of words or bytes, indexed by integers called addresses. Modern machines – say, designed during the last 20 years – have in addition tended to support directly the notion of a function call stack. Furthermore, all popular machines have some important facilities – such as input-output – that do not fit well into the conventional byte- or word-oriented model of memory or computation. These facilities may require special machine instructions or access to “memory” locations with peculiar semantics. Either way, from a higher-level language point of view, the use of these facilities is messy and machine-architecture-specific.

C is by far the most successful language providing the programmer with a programming model that closely matches the machine model. C provides language-level and machine-architecture-independent notions that directly map to the key hardware notions: characters for using bytes, integers for using words, pointers for using the addressing mechanisms, functions for program abstraction, and an absence of constraining language features so that the programmer can manipulate the inevitable messy hardware-specific details. The net effect has been that C is relatively easy to learn and use in areas where some knowledge of the real machine is a benefit. Moreover, C is easy enough to implement that it has become almost universally available.

STORAGE

In C and C++, there are three fundamental ways of using memory:

Static memory, in which an object is allocated by the linker for the duration of the program. Global variables, *static* class members, and *static* variables in functions are allocated in static memory. An object allocated in static memory is constructed once and persists to the end of the program. Its address does not change while the program is running. Static objects can be a problem in programs using threads (shared-address-space concurrency) because they are shared and require locking for proper access.

Automatic memory, in which function arguments and local variables are allocated. Each entry into a function or a block gets its own copy. This kind of memory is automatically created and destroyed; hence the name automatic memory. Automatic memory is also said “to be on the stack.”

Free store, from which memory for objects is explicitly requested by the program and where a program can free memory again once it is done with it (using the *new* and *delete* operators).

When a program needs more free store, *new* requests it from the operating system.

Typically, the free store (also called *dynamic memory* or *the heap*) grows throughout the lifetime of a program because no memory is ever returned to the operating system for use by other programs.

For example:

```
int g = 7; // global variable, statically allocated void f()
{
    int loc = 9; // local variable, stack allocated in t* p = new int;
    //
    variable allocated on free store
    // ...
    delete p; // return variable pointed to by p for possible reuse
}
```

2.1 Compile, Link & Execute

Traditionally, a C or a C++ program consists of a number of source files that are individually compiled into object files. These object files are then linked together to produce the executable form of the program.

Each separately compiled program fragment must contain enough information to allow it to be linked together with other program fragments. Most language rules are checked by the compiler as it compiles an individual source file (translation unit). The linker checks to ensure that names are used consistently in different compilation units and that every name used actually refers to something that has been properly defined. The typical C++ runtime environment performs few checks on the executing code. A programmer who wants run-time checking must provide the tests as part of the source code.

C++ interpreters and dynamic linkers modify this picture only slightly by postponing some checks until the first use of a code fragment.

2.2 Type Checking

C++ provides and relies on *static* type checking. That is, most language rules are checked by the compiler before a program starts executing. Each entity in a program has a type and must be used in accordance with its type. For example:

```
int  f(double); // f is a function that takes a double-precision
floating point

// argument and returns an integer

float  x = 2. 0; // x is a single- precision floating point object
string s = "2";  // s is a string of characters

int  i = f(x);  // i is an integer
```

2.3 Concrete Types

Small heavily used abstractions are common in many applications. Examples are characters, integers, floating point numbers, complex numbers, points, pointers, coordinates, transforms, (*pointer ,offset*) pairs, dates, times, ranges, links, associations, nodes, (*value ,unit*) pairs, disc locations, source code locations, *BCD* characters, currencies, lines, rectangles, scaled fixed point numbers, numbers with fractions, character strings, vectors, and arrays. Every application uses several of these; a few use them heavily. A typical application uses a few directly and many more indirectly from libraries.

The designer of a general-purpose programming language cannot foresee the detailed needs of every application. Consequently, such as language must provide mechanisms for the user to define such small concrete types. It was an explicit aim of C++ to support the definition and efficient use of such user-defined data types very well. They were seen as the foundation of elegant programming. The simple and mundane is statistically far more significant than the complicated and sophisticated.

Many concrete types are frequently used and subject to a variety of constraints. Consequently, the language facilities supporting their construction emphasizes flexibility and uncompromising time and space efficiency. Where more convenient, higher-level, or safer types are needed, such types can be built on top of simple efficient types. The opposite – building uncompromisingly efficient types on top of more complicated “higher-level” types – cannot be done. Consequently, languages that do not provide facilities for efficient user-defined concrete types need to provide more built-in types, such as lists, strings, and vectors supported by special language rules.

C++ STANDARD LIBRARY

The standard library provides:

- [1] Basic run-time language support (e.g., for allocation and run-time type information).
- [2] The C standard library (with very minor modifications to minimize violations of the type system).
- [3] Strings and I/O streams (with support for international character sets and localization).
- [4] A framework of containers (such as vector, list, and map) and algorithms using containers (such as general traversals, sorts, and merges).
- [5] Support for numerical computation (complex numbers plus vectors with arithmetic operations, BLAS-like and generalized slices, and semantics designed to ease optimization).

The main criterion for including a class in the library was that it would somehow be used by almost every C++ programmer (both novices and experts), that it could be provided in a general form that did not add significant overhead compared to a simpler version of the same facility, and that simple uses should be easy to learn. Essentially, the C++ standard library provides the most common fundamental data structures together with the fundamental algorithms used on them.

The framework of containers, algorithms, and iterators is commonly referred to as the STL. It is primarily the work of Alexander Stepanov [Stepanov,1994].

3.1 Strings & I/O

Strings and input/output operations are not provided directly by special language constructs in C++. Instead, the standard library provides string and I/O types. For example:

```

#include<string>          // make standard strings available
#include< iostream> // make standard I/O available

int main()
{
    using namespace std;

    string name;

    cout << " Please enter your name: ";    // prompt the user
    cin >> name;        // read a name

    cout << " Hello, " << name << '\n';    // output the name
    followed by a newline return 0;
}

```

This example uses the standard input and output streams, *cin* and *cout* with their operators *>>* (“get from”) and *<<* (“put to”).

The I/O streams support a variety of formatting and buffering facilities, and strings support common string operations such as concatenation, insertion, and extraction of characters and strings. Both streams and strings can be used with characters of any character set.

The standard library facilities can be – and usually are – implemented using only facilities available to all users. Consequently, where the standard facilities happen to be inadequate, a user can provide equally elegant alternatives

3.2 Vector Arithmetic

The standard *vector* from §6.2 was designed to be a general mechanism for holding values, to be flexible, and to fit into the architecture of containers, iterators, and algorithms. However, it does not support mathematical vector operations. Adding such operations to *vector* would be easy, but its generality and flexibility precludes optimizations that are often considered essential for serious numerical work. Consequently, the standard library provides a vector, called *valarray*, that is less general and more amenable to optimization for numerical computation:

```

template<class T> class valarray {

// ...

T& operator[] (size_t);

// ...

};

```

The type *size_t* is the unsigned integer type that the implementation uses for array indices.

The usual arithmetic operations and the most common mathematical functions are supported for *valarrays*. For example:

```

template<class T> valarray<T> abs(const valarray<T>&); // absolute value

void f(const valarray<double>& a1, const valarray<double>& a2)

{

    valarray<double> a = a1*3.14+a2/a1;

    a += a2*3.14;

    valarray<double> aa = abs(a);

    double d = a2[7];

    // ...

}

```

OBJECT ORIENTED PROGRAMMING

4.1 Basic Concept on OOPs –

1. Objects
2. Classes
3. Data abstraction and encapsulation
4. Inheritance
5. Polymorphism
6. Dynamic binding
7. Message passing

4.1.1 OBJECTS

Objects are the basic run-time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program must handle. The fundamental idea behind object oriented approach is to combine both data and function into a single unit and these units are called objects.

The term objects means a combination of data and program that represent some real word entity. For example: consider an example named Amit; Amit is 25 years old and his salary is 2500.

The Amit may be represented in a computer program as an object. The data part of the object would be (name: Amit, age: 25, salary: 2500)

The program part of the object may be collection of programs (retrieve of data, change age, change of salary). In general even any user –defined type-such as employee may be used. In the Amit object the name, age and salary are called attributes of the object.

4.1.2 Class

A group of objects that share common properties for data part and some program part are collectively called as class.

In C ++ a class is a new data type that contains member variables and member functions that

operate on the variables.

4.1.3 Data Abstraction

Abstraction refers to the act of representing essential features without including the background details or explanations. Classes use the concept of abstraction and are defined as size, width

and cost and functions to operate on the attributes

4.1.4 Data Encapsulation

The wrapping up of data and function into a single unit (called class) is known as encapsulation. The data is not accessible to the outside world and only those functions which are wrapped in the class can access it. These functions provide the interface between the objects data and the program.

4.1.5 Inheritance

Inheritance is the process by which objects of one class acquire the properties of another class. In the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. This is possible by designing a new class which will have the combined features of both the classes

4.1.6 Polymorphism

Polymorphism means the ability to take more than one form. An operation may exhibit different instances. The behaviour depends upon the type of data used in the operation.

A language feature that allows a function or operator to be given more than one definition.

The types of the arguments with which the function or operator is called determines which definition will be used.

Overloading may be operator overloading or function overloading.

It is able to express the operation of addition by a single operator say '+'. When this is possible you use the expression $x + y$ to denote the sum of x and y , for many different types of x and y ; integers, float and complex no. You can even define the $+$ operation for two

strings to mean the concatenation of the strings.

4.2 Benefits of OOPs –

Oop offers several benefits to both the program designer and the user. Object-oriented contributes to the solution of many problems associated with the development and quality of software product. The principal advantages are :

1. Through inheritance we can eliminate redundant code and extend the use of existing classes.
2. We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
3. This principle of data hiding helps the programmer to build secure programs that can't be invaded by code in other parts of the program.
4. It is possible to have multiple instances of an object to co-exist with out any interference.
5. It is easy to partition the work in a project based on objects.
6. Object-oriented systems can be easily upgraded from small to large systems.
7. Message passing techniques for communication between objects makes the interface description with external systems much simpler.
8. Software complexity can be easily managed

4.3 Applications of OOPs –

The most popular application of oops up to now, has been in the area of user interface design such as windows. There are hundreds of windowing systems developed using oop techniques.

Real business systems are often much more complex and contain many more objects with complicated attributes and methods. Oop is useful in this type of applications because it can simplify a complex problem. The promising areas for application of oop includes.

1. Real – Time systems.
2. Simulation and modeling
3. Object oriented databases.

4. Hypertext hypermedia and expertext.
5. AI and expert systems.
6. Neural networks and parallel programming.
7. Decision support and office automation systems.
8. CIM / CAM / CAD system

FUNCTION IN C++

5.1 The main() Function –

ANSI does not specify any return type for the main () function which is the starting point for the execution of a program . The definition of main() is :-

```
main()  
{  
//main program statements  
}
```

This is property valid because the main () in ANSI C does not return any value. In C++, the main () returns a value of type int to the operating system. The functions that have a return value should use the return statement for terminating.

The main () function in C++ is therefore defined as follows.

```
int main( )  
{  
-----  
-----  
return (0)  
}
```

Since the return type of functions is int by default, the key word int in the main() header is optional

5.2 Inline Function –

To eliminate the cost of calls to small functions C++ proposes a new feature called inline function.

An inline function is a function that is expanded inline when it is invoked .That is the compiler replaces the function call with the corresponding function code.

The inline functions are defined as follows:-

```
inline function-header
{
function body;
}
```

Example:

```
inline double cube (double a)
{
return (a*a*a) ;
}
```

The above inline function can be invoked by statements like

```
c=cube(3.0);
```

```
d=cube(2.5+1.5);
```

remember that the inline keyword merely sends a request, not a command to the compiler.

The compiler may ignore this request if the function definition is too long or too complicated and compile the function as a normal function

Some of the situations where inline expansion may not work are:

1. For functions returning values if a loop, a switch or a go to exists.
2. for function s not returning values, if a return statement exists.
3. if functions contain static variables.
4. if inline functions are recursive,

Example:

```
#include<iostream.h>
#include<stdio.h>
inline float mul(float x, float y)
{
return (x*y) ;
}
inline double div(double p, double q)
{
return (p/q) ;
}
```

```

}
main( )
{
float a=12.345;
float b=9.82;
cout << mul(a,b) << endl;
cout << div(a,b) << endl;
}

```

Output:
121.227898
1.257128

5.3 Default Argument –

C++ allows us to call a function with out specifying all its arguments. In such cases, the function assigns a default value to the parameter which does not have a matching arguments in the function call. Default values are specified when the function is declared .The compiler looks at the prototype to see how many arguments a function uses and alerts the program for possible default values.

Example: float amount (float principle, int period ,float rate=0.15);

The default value is specified in a manner syntactically similar to a variable initialization .The above prototype declares a default value of 0.15 to the argument rate. A subsequent function call like value=amount(5000,7); //one argument missing passes the value of 5000 to principle and 7 to period and then lets the function, use default value of 0.15 for rate.

The call:- value=amount(5000,5,0.12);

//no missing argument passes an explicit value of 0.12 rate.

One important point to note is that only the trailing arguments can have default values. That is, we must add default from right to left .We cannot provide a default to a particular argument in the middle of an argument list.

Example:- int mul(int i, int j=5,int k=10);//illegal

int mul(int i=0,int j, int k=10);//illegal

int mul(int i=5,int j); //illegal

int mul(int i=2,int j=5,int k=10);//illegal

Default arguments are useful in situation whose some arguments always have the some value.

For example, bank interest may retain the same for all customers for a particular period of deposit.

Example:

```
#include<iostream.h>
#include<stdio.h>
mainQ
{
    float amount;
    float value(float p,int n,float r=0.15); void
    printline(char ch='*',int len=40); printline( );
    amount=value(5000.00,5);
    cout<<"\n final value="<<amount<<endl;
    printline('=');
    //function definitions
    float value (float p,int n, float r)
    {
        float si;
        si=(p*n*r)/100;
        return(si);
    }
    void printline (char ch,int len)
    {
        for(int i=1;i<=len;i++)
            cout<<ch<<endl;
    }
}
```

CLASS

Class is a group of objects that share common properties and relationships .In C++, a class is a new data type that contains member variables and member functions that operates on the variables.

A class is defined with the keyword class. It allows the data to be hidden, if necessary from external use. When we defining a class, we are creating a new abstract data type that can be treated like any other built in data type.

Generally a class specification has two parts:-

a) Class declaration

b) Class function definition

the class declaration describes the type and scope of its members. The class function definition describes how the class functions are implemented.

Syntax:-

```
class class-name
{
    private:
        variable
        declarations;
        function
        declaration ;
    public:
        variable
        declarations;
        function
        declaration;
};
```

The members that have been declared as private can be accessed only from within the class. On the other hand , public members can be accessed from outside the class also. The data hiding is the key feature of oops. The use of keywords private is optional by default, the members of a class are private.

The variables declared inside the class are known as data members and the functions are known as members mid the functions. Only the member functions can have access to the private data members and private functions. However, the public members can be accessed from the outside the class. The binding of data and functions together into a single class type variable is referred to as encapsulation.

Syntax:-

```
class
item
{
    int
    member;
public: float
c: cost;
    void getldata (int
a ,float b); void putdata
(void);
```

The class item contains two data members and two function members, the data members are private by default while both the functions are public by declaration. The function getdata() can be used to assign values to the member variables member and cost, and putdata() for displaying their values . These functions provide the only access to the data members from outside the class.

6.1 Defining member function –

Member can be defined in two places

- Outside the class definition
- Inside the class function

6.2 Outside the Class Definition –

Member function that are declared inside a class have to be defined separately outside the class. Their definition are very much like the normal functions.

An important difference between a member function and a normal function is that a member function incorporates a membership. Identify label in the header. The ‘label’ tells the compiler which class the function belongs to.

The member ship label `class-name ::` tells the compiler that the function function - name belongs to the class `class-name` . That is the scope of the function is restricted to the classname specified in the header line. The `::` symbol is called scope resolution operator.

Example:

```
void item :: getdata (int a , float b )
{
    number=a;
    cost=b;
}

void item :: putdata ( void)
{
    cout<<"number="<<number<<endl;
    cout<<"cost="<<cost<<endl;
}
```

The member function have some special characteristics that are often used in the program development.

- Several different classes can use the same function name. The "membership label" will resolve their scope, member functions can access the private data of the class. A non-member function can't do so.
- A member function can call another member function directly, without using the dot operator

6.3 A C++ Program with Class –

```
# include< iostream.
h> class item
{
    int
    number;
publi float
c: cost;
void getdata ( int a , float b);
```

```

void putdala ( void)
{
    cout<<"number:"<<number<<endl;
    cout<<"cost : "<<cost<<endl;
}
};

void item :: getdata (int a , float b)
{
    number=a;
    cost=b;
}

main
( )
{
    item x;
    cout<<"\nobjectx"<<endl;
    x.
    getdata( 100,299.95)
    ; x .putdata();
    item y;
    cout<<"\n object y"<<endl;
    y. getdata(200,175.5);
}
    y. putdata();

```

Output:

object x

number 10040 P.T.O

cost=299.950012

object -4

cost=175.5

6.4 Private Member Function –

Although it is a normal practice to place all the data items in a private section and all the functions in public, some situations may require contain functions to be hidden from the outside calls. Tasks such as deleting an account in a customer file or providing increment to and employee are events of serious consequences and therefore the functions handling such tasks should have restricted access. We can place these functions in the private section.

A private member function can only be called by another function that is a member of its class. Even an object can not invoke a private function using the dot operator

Example –

```
#include<iostream.h>
> class distance
{
    private:
        int feet;
        float
        inches;

    public:
        void setdist ( int ft, float in)
        {
            feet=ft;
            inches=i
            n;
        }
    void getdist()
    {
        cout<<"enter
        feet:"; cin>>feet;
        cout<<"enter
        inches:";
        cin>>inches;
    }
    void showdist()
```

```

{
    cout<< feet<<"_ "inches « endl;
}

};

void main( )
{
    distance d1,d2;
    d1.setdist(1
    1,6.25);
    d2.getdata();
    cout<<endl<<"dist:"<<d 1 .showdist();
    cout<<"\n"<<"dist2:";
    d2.showdist();
}

```

Output:

enter feet: 12

enter inches: 6.25

dist 1:"11'- 6.1.5"

dist 2: 12'- 6.

Constructors

A constructor is a special member function whose task is to initialize the objects of its class. It is special because its name is the same as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class.

A constructor is declared and defined as follows:

```
//class with a constructor
class integer
{
    int m,n;
public:
    integer() void; //constructor declared
    -----
    -----
};
integer :: integer(void)
{
    m=0;
    n=0;
}
```

When a class contains a constructor like the one defined above it is guaranteed that an object created by the class will be initialized automatically.

For example:-

```
Integer int1; //object int 1 created
```

This declaration not only creates the object int1 of type integer but also initializes its data members m and n to zero.

A constructor that accepts no parameter is called the default constructor. The default

constructor for class A is A :: A(). If no such constructor is defined, then the compiler supplies a default constructor.

Therefore a statement such as :-

A a ;//invokes the default constructor of the compiler to create the object "a" ;

Invokes the default constructor of the compiler to create the object a.

The constructor functions have some characteristics:-

- They should be declared in the public section .
- They are invoked automatically when the objects are created.
- They don't have return types, not even void and therefore they cannot return values.
- They cannot be inherited , though a derived class can call the base class constructor .
- Like other C++ function , they can have default arguments,
- Constructor can't be virtual.
- An object with a constructor can't be used as a member of union.

7.1 Example of Default Constructor -

```
#include<iostream.
h>
#include<conio.h>
class abc
{
private:
    char nm[];
public:
c:    abc ( )

{
    cout<<"enter your name:";
    cin>>nm;

}
void display( )
{
```

```

        cout<<nm;
    }
};
int main( )
{
    clrscr( );
    abc d;
    d.display
    ( );
    getch( )
    ;
    return(
    0);
}

```

7.2 Parametrized Constructor –

the constructors that can take arguments are called parameterized constructors. Using parameterized constructor we can initialize the various data elements of different objects with different values when they are created

Example:

```

class
integer
{
    int m,n;
public:
    integer( int x, int y);
    -----
    -----
};
integer:: integer (int x, int y)
{
    m=x;n=y;
}

```

the argument can be passed to the constructor by calling the constructor implicitly.

```
integer int 1 = integer(0,100); // explicit call  
integer int 1(0,100); //implicite call
```

7.3 Copy Constructor –

A copy constructor is used to declare and initialize an object from another object.

Example:-

the statement

```
integer 12(11);
```

would define the object 12 and at the same time initialize it to the values of 11.

Another form of this statement is : `integer 12=11;`

The process of initialization through a copy constructor is known as copy initialization.

TYPE CONVERSIONS

In a mixed expression constants and variables are of different data types. The assignment operations causes automatic type conversion between the operand as per certain rules.

The type of data to the right of an assignment operator is automatically converted to the data type of variable on the left.

Consider the following example:

```
int x;  
float y = 20.123;  
x=y ;
```

This converts float variable y to an integer before its value assigned to x. The type conversion is automatic as far as data types involved are built in types. We can also use the assignment operator in case of objects to copy values of all data members of right hand object to the object on left hand. The objects in this case are of same data type. But of objects are of different data types we must apply conversion rules for assignment.

There are three types of situations that arise where data conversion are between incompatible types.

1. Conversion from built in type to class type.
2. Conversion from class type to built in type.
3. Conversion from one class type to another.

8.1 Basic To Class Type –

A constructor was used to build a matrix object from an int type array. Similarly, we used another constructor to build a string type object from a char* type variable. In these examples constructors performed a defacto type conversion from the argument's type to the constructor's class type

Consider the following constructor:

```
string :: string (char*a)  
{
```

```

length = strlen (a);
name=new
char[len+1]; strcpy
(name,a);
}

```

This constructor builds a string type object from a char* type variable a. The variables length and name are data members of the class string. Once you define the constructor in the class string, it can be used for conversion from char* type to string type.

Example –

```
string si , s2;
```

```
char* name1 = “Good Morning”;
```

```
char* name2 = “ STUDENTS” ;
```

```
s1 = string(name1);
```

```
s2 = name2;The program statement
```

```
si = string (name1);
```

first converts name 1 from char* type to string type and then assigns the string type values to the object s1. The statement

```
s2 = name2;
```

performs the same job by invoking the constructor implicitly.

Consider the following example

```

class time
{
    int hours;
    int minutes;
public:
    c:
        time (int t) // constructor
    {
        hours = t / 60; //t is inputted in minutes
        minutes = t % 60;
    }
};

```

In the following conversion statements :

```
time Tl; //object Tl created
```

```
int period = 160;
```

```
Tl = period; //int to class type
```

The object Tl is created. The variable period of data type integer is converted into class type time by invoking the constructor. After this conversion, the data member hours of Tl will have value 2 and minutes will have a value of 40 denoting 2 hours and 40 minutes.

Note that the constructors used for the type conversion take a single argument whose type is to be converted.

In both the examples, the left-hand operand of = operator is always a class object. Hence, we can also accomplish this conversion using an overloaded = operator.

8.2 Class to Basic Type

The constructor functions do not support conversion from a class to basic type. C++ allows us to define an overloaded casting operator that convert a class type data to basic type. The general form of an overloaded casting operator function, also referred to as a conversion function, is:

```
operator typename ( )  
{  
    //Program statement .  
}
```

This function converts a class type data to typename. For example, the operator double() converts a class object to type double, in the following conversion function:

```
vector::operator double ( )  
{  
    double sum = 0 ;  
    for(int I = 0;  
        I < v.size();  
        I++)  
        sum = sum + v[I] * v[I] ; //scalar  
    magnitude return sqrt(sum);  
}
```

The casting operator should satisfy the following conditions.

- It must be a class member.
- It must not specify a return type.
- It must not have any arguments. Since it is a member function, it is invoked by the object and therefore, the values used for, Conversion inside the function belongs to the object that invoked the function. As a result function does not need an argument.

In the string example discussed earlier, we can convert the object string to char* as follows:

```
string::operator char*( )
{
    return (str) ;
}
```

8.3 One Class to Another Class Type

We have just seen data conversion techniques from a basic to class type and a class to basic type. But sometimes we would like to convert one class data type to another class type

Example-

Obj1 = Obj2 ; //Obj1 and Obj2 are objects of different classes.

Obj1 is an object of class one and Obj2 is an object of class two. The class two type data is converted to class one type data and the converted value is assigned to the Obj1. Since the conversion takes place from class two to class one, two is known as the source and one is known as the destination class.

Such conversion between objects of different classes can be carried out by either a constructor or a conversion function. Which form to use, depends upon where we want the type conversion function to be located, whether in the source class or in the destination class.

We studied that the casting operator function

Operator typename()

Converts the class object of which it is a member to typename. The type name may be a built-in type or a user defined one(another class type) . In the case of conversions between objects, typename refers to the destination class. Therefore, when a class needs to be converted, a casting operator function can be used. The conversion takes place in the source

class and the result is given to the destination class object.

Let us consider a single-argument constructor function which serves as an instruction for converting the argument's type to the class type of which it is a member. The argument belongs to the source class and is passed to the destination class for conversion. Therefore the conversion constructor must be placed in the destination class.

When a conversion using a constructor is performed in the destination class, we must be able to access the data members of the object sent (by the source class) as an argument. Since data members of the source class are private, we must use special access functions in the source class to facilitate its data flow to the destination class.

Consider the following example of an inventory of products in a store. One way of keeping record of the details of the products is to record their code number, total items in the stock and the cost of each item. Alternatively we could just specify the item code and the value of the item in the stock. The following program uses classes and shows how to convert data of one type to another.

```
#include<iostream.  
h>  
#include<conio.h>  
class stock2;  
class stock1  
{  
    int code, item;  
    float price;  
public:  
    stock1 (int a, int b, float c)  
    {  
        code=a;  
        item=b;  
        price=  
        c;  
    }  
void disp( )  
{cout<<"code"<<code <<"\n";
```

```

        cout<<"Items"<<item <<"\n";
        cout<<"Price per item Rs . " <<price <<"\n";
    }

    int getcode( )
    {return
    code; } int
    getitem( )
    {return item; }
int getprice( )
    {return price;}operator float( )
{
return ( item*price );
}
};
class stock2
{
    int
    code;
public float
c:    val;

    stock2()
    {
        code=0; val=0;
    }
    stock2(int x, float y)
    {
        code=x; val=y;
    }
void disp( )
{
    cout<< "code"<<code << "\n";
    cout<< "Total Value Rs . " <<val <<"\n"
}

```

```

stock2 (stock1 p)
{
    code=p . getcode ( ) ;
    val=p.getitem( ) * p.
    getprice ( ) ;
}
};
void main ( )
{ '
    Stock1 i1(101, 10,125.0);
    Stock2 i2;
    float
    tot_val;
    tot_val=i1
    ; i2=i1 ;
    cout<<" Stock Details-stock1-type"
    <<"\n"; i1 . disp ( ) ;
    cout<<" Stock value"<<"\n";
    cout<< tot_val<<"\n";
    cout<<" Stock Details-stock2-
    type"<< "\n"; i2 .disp( ) ;
    getch ( ) ;
}

```

You should get the following output.

Stock Details-stock1-type

code 101

Items 10Price per item Rs. 125

Stock value

1250

Stock Details-stock2-type

code 10 1

Total Value Rs. 1250

INHERITANCE

Reaccessability is yet another feature of OOP's. C++ strongly supports the concept of reusability. The C++ classes can be used again in several ways. Once a class has been written and tested, it can be adopted by another programmers. This is basically created by defining the new classes, reusing the properties of existing ones. The mechanism of deriving a new class from an old one is called 'INHERITANCE'. This is often referred to as IS-A' relationship because very object of the class being defined "is" also an object of inherited class. The old class is called 'BASE' class and the new one is called 'DERIVED' class.

9.1 Defining Derived Classes

A derived class is specified by defining its relationship with the base class in addition to its own details. The general syntax of defining a derived class is as follows:

```
class d classname : Access specifier baseclass name
{
    —
    — // members of derived class
};
```

The colon indicates that the a-class name is derived from the base class name. The access specifier or the visibility mode is optional and, if present, may be public, private or protected. By default it is private. Visibility mode describes the status of derived features e.g.

```
class xyz //base class
{
    members of xyz
};
class ABC : public xyz //public derivation
{
    members of ABC
};
class ABC : XYZ //private derivation (by default)
```

```
{
    members of ABC
};
```

In the inheritance, some of the base class data elements and member functions are inherited into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capabilities of the existing classes, becomes a very powerful tool for incremental program development.

9.2 Single Inheritance –

When a class inherits from a single base class, it is known as single inheritance. Following program shows the single inheritance using public derivation.

```
#include<iostream.
h>
#include<conio.h>
class worker
{
    Int age;
    char name [10];
public:
void get ( );
};
void worker : : get ( )
{
    cout <<"yout name
please" cin >> name;
    cout <<"your age
please" ; cin >> age;
}
void worker :: show ( )
{
    cout <<"In My name is :"<<name<<"In My age is :"<<age;
```

```

}
class manager :: public worker //derived class (publicly)
{
    int now;
public:
void get
( ) ; void
show ( ) ;
};
void manager : : get ( )
{
    worker : : get ( ) ; //the calling of base
    class input fn. cout << "number of workers
    under you";
    cin >> now;
    cin>>name>>age;
} ( if they were
public ) void
manager :: show ( )
{
    worker :: show ( ) ; //calling of base
    class o/p fn. cout <<"in No. of workers
    under me are: " << now;
}
main ( )
{
    clrscr
    ( ) ;
    worker W1;
    manager M1;
    M1 .get
    ( ) ;
    M1.show ( ) ;
}

```

If you input the following to this program:

Your name please

Ravinder

Your age please

27

number of workers under you

30Then the output will be as follows:

My name is : Ravinder

My age is : 27

No. of workers under me are : 30

9.3 Making a Private Member Inheritable

Basically we have visibility modes to specify that in which mode you are deriving the another class from the already existing base class. They are:

9.3.1 Private:

when a base class is privately inherited by a derived class, 'public members' of the base class become private members of the derived class and therefore the public members of the base class can be accessed by its own objects using the dot operator. The result is that we have no member of base class that is accessible to the objects of the derived class

9.3.2 Public

On the other hand, when the base class is publicly inherited, 'public members' of the base class become 'public members' of derived class and therefore they are accessible to the objects of the derived class

9.3.3 Protected

C++ provides a third visibility modifier, protected, which serve a little purpose in the inheritance. A member declared as protected is accessible by the member functions within its class and any class immediately derived from it. It cannot be accessed by functions outside these two classes.

The private and protected members of a class can be accessed by:

a. A function i.e. friend of a class.

- b. A member function of a class that is the friend of the class.
- c. A member function of a derived class.

The below mentioned table summarizes how the visibility of members undergo modifications when they are inherited

Base Class Visibility	Derived Class Visibility		
	Public	Private	Protected
Private	X	X	X
Public	Public	Private	Protected
Protected	Protected	Private	Protected

Table 9.1: Inheritance Visibility

9.4 Multilevel Inheritance

When the inheritance is such that, the class A serves as a base class for a derived class B which in turn serves as a base class for the derived class C. This type of inheritance is called 'MULTILEVEL INHERITANCE'. The class B is known as the 'INTERMEDIATE BASE CLASS' since it provides a link for the inheritance between A and C. The chain ABC is called 'INHERITANCE*PATH' for e.g.

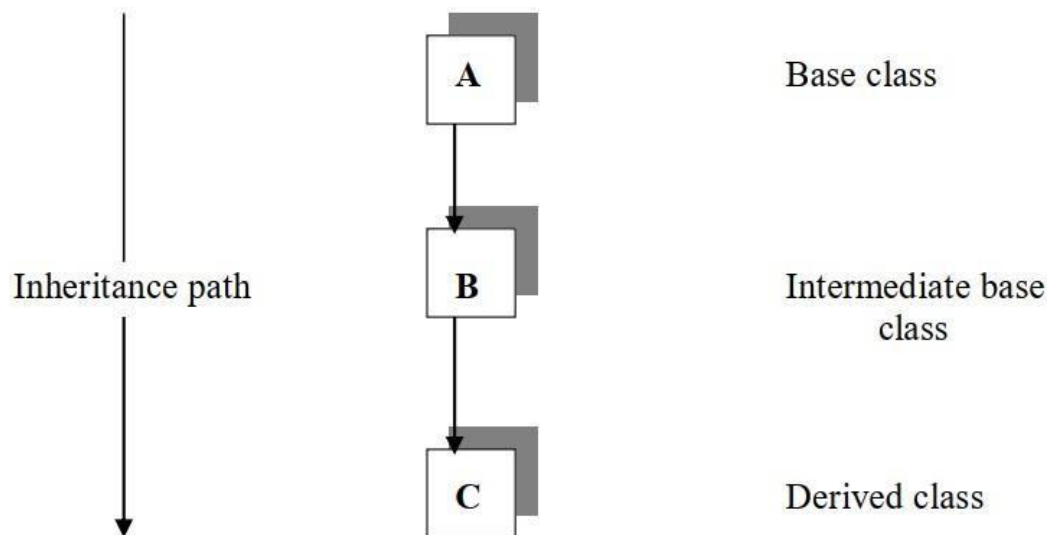


Figure 9.1: Multilevel Inheritance

The declaration for the same would be:

```
Class A
{
//body
}
Class B : public A
{
//body
}
Class C : public B
{
//body
}
```

A class can inherit the attributes of two or more classes. This mechanism is known as ‘MULTIPLE INHERITANCE’. Multiple inheritance allows us to combine the features of several existing classes as a starting point for defining new classes. It is like the child inheriting the physical feature of one parent and the intelligence of another. The syntax of the derived class is as follows:

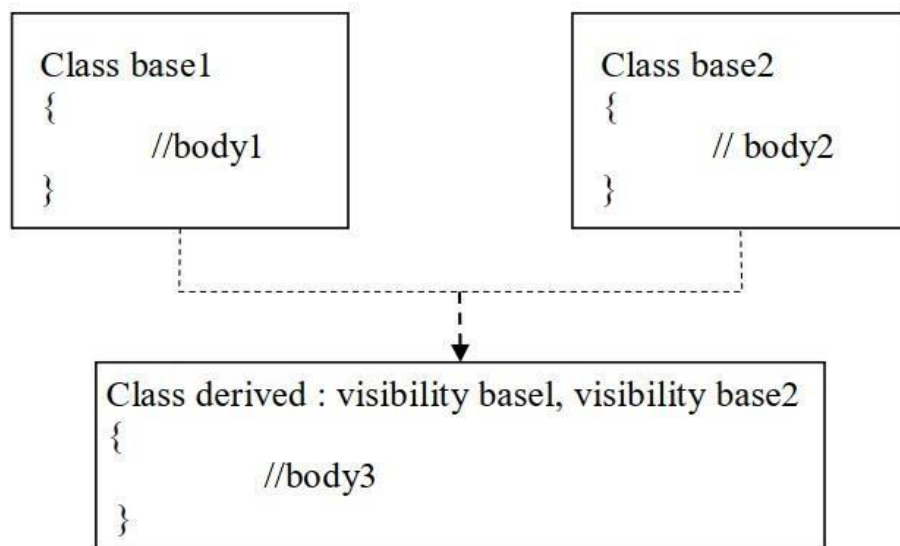


Figure 9.2: Multiple Inheritance

9.5 Hierarchical Inheritance

Another interesting application of inheritance is to use it as a support to a hierarchical design of a class program. Many programming problems can be cast into a hierarchy where certain features of one level are shared by many others below that level for e.g.

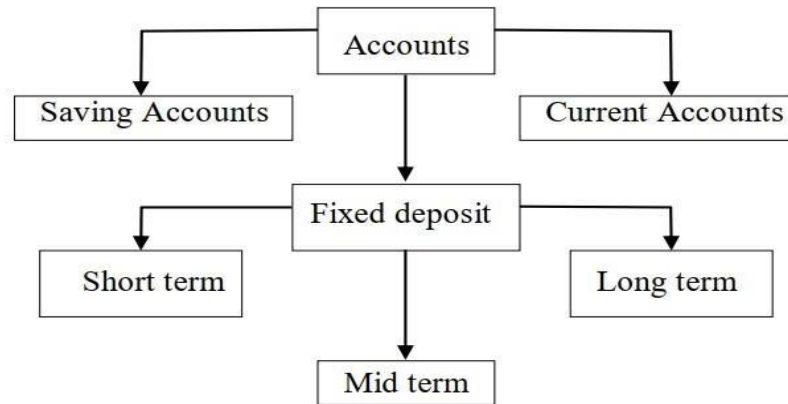


Figure 9.3: Hierarchical Inheritance

In general the syntax is given as

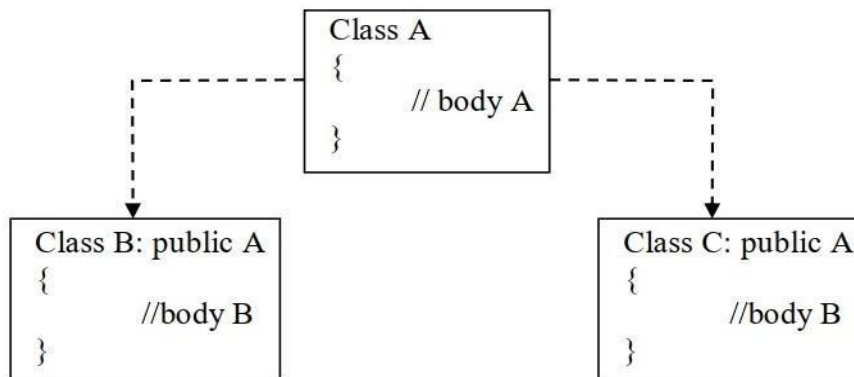


Figure 9.4 Hierarchical Inheritance Approach

In C++, such problems can be easily converted into hierarchies. The base class will include all the features that are common to the subclasses. A sub-class can be constructed by inheriting the features of base class and so on.

9.6 Hybrid Inheritance

There could be situations where we need to apply two or more types of inheritance to design a program. Basically Hybrid Inheritance is the combination of one or more types of the inheritance.

POLYMORPHISM

10.1 Introduction

When an object is created from its class, the member variables and member functions are allocated memory spaces. The memory spaces have unique addresses. Pointer is a mechanism to access these memory locations using their address rather than the name assigned to them. You will study the implications and applications of this mechanism in detail in this chapter.

Pointer is a variable which can hold the address of a memory location rather than the value at the location. Consider the following statement

```
int num =84;
```

This statement instructs the compiler to reserve a 2-byte of memory location and puts the value 84 in that location. Assume that the compiler allocates memory location 1001 to num. Diagrammatically, the allocation can be shown as

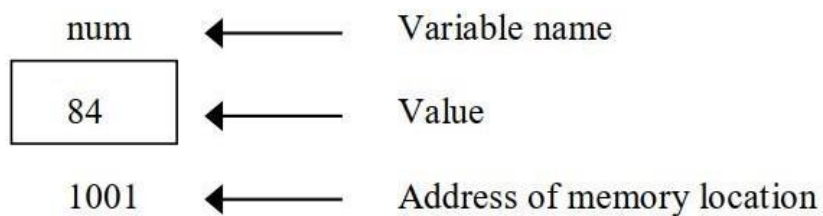


Figure 10.1: Memory Representation of A Variable

As the memory addresses are themselves numbers, they can be assigned to some other variable. For example, ptr be the variable to hold the address of variable num.

10.2 Pointers to Objects

An object of a class behaves identically as any other variable. Just as pointers can be defined in case of base C++ variables so also pointers can be defined for an object type. To create a pointer variable for the following class

```
class employee {  
    int code;
```

```

        char name [20] ;
public:
c:      inline void getdata
        ( )= 0 ; inline void
        display ( )= 0 ;
};

```

The following codes is written

```
employee *abc;
```

This declaration creates a pointer variable abc that can point to any object of employee type.

10.3 this Pointer

C++ uses a unique keyword called "this" to represent an object that invokes a member function. 'this' is a pointer that points to the object for which this function was called. This unique pointer is called and it passes to the member function automatically. The pointer this acts as an implicit argument to all the member function, for e.g

```

class ABC
{
    int a ;
    ----
    ----
};

```

The private variable 'a' can be used directly inside a member function, like

```
a=123;
```

We can also use the following statement to do the same job.

```
this → a = 123
```

10.4 Virtual Functions

Virtual functions, one of advanced features of OOP is one that does not really exist but it appears real in some parts of a program. This section deals with the polymorphic features

which are incorporated using the virtual functions.

The general syntax of the virtual function declaration is:

```
class use_detined_name{
private:
e:
public:
    virtual    return_type    function_name1
    (arguments);    virtual    return_type
    function_name2(arguments);    virtual
    return_type function_name3( arguments);
    -----
};
```

To make a member function virtual, the keyword `virtual` is used in the methods while it is declared in the class definition but not in the member function definition. The keyword `virtual` precedes the return type of the function name. The compiler gets information from the keyword `virtual` that it is a virtual function and not a conventional function declaration.

Remember that the keyword `virtual` should not be repeated in the definition if the definition occurs outside the class declaration. The use of a function specifier `virtual` in the function definition is invalid.

EXCEPTION HANDLING

Exception refers to unexpected condition in a program. The unusual conditions could be faults, causing an error which in turn causes the program to fail. The error handling mechanism of c++ is generally referred to as exception handling.

Generally, exceptions are classified into synchronous and asynchronous exceptions. The exceptions which occur during the program execution, due to some fault in the input data or technique that is not suitable to handle the current class of data. within a program is known as synchronous exception.

Example:

errors such as out of range, overflow, underflow and so on.

The exceptions caused by events or faults unrelated to the program and beyond the control of program are asynchronous exceptions.

For example, errors such as keyboard interrupts, hardware malfunctions, disk failure and so on.

11.1 Exception Handling Model

When a program encounters an abnormal situation for which it is not designed, the user may transfer control to some other part of the program that is designed to deal with the problem. This is done by throwing an exception. The exception handling mechanism uses three blocks: try, throw and catch. The try block must be followed immediately by a handler, which is a catch block. If an exception is thrown in the try block the program control is transferred to the appropriate exception handler. The program should attempt to catch any exception that is thrown by any function. The relationship of these three exceptions handling constructs called the exception handling model is shown in figure:

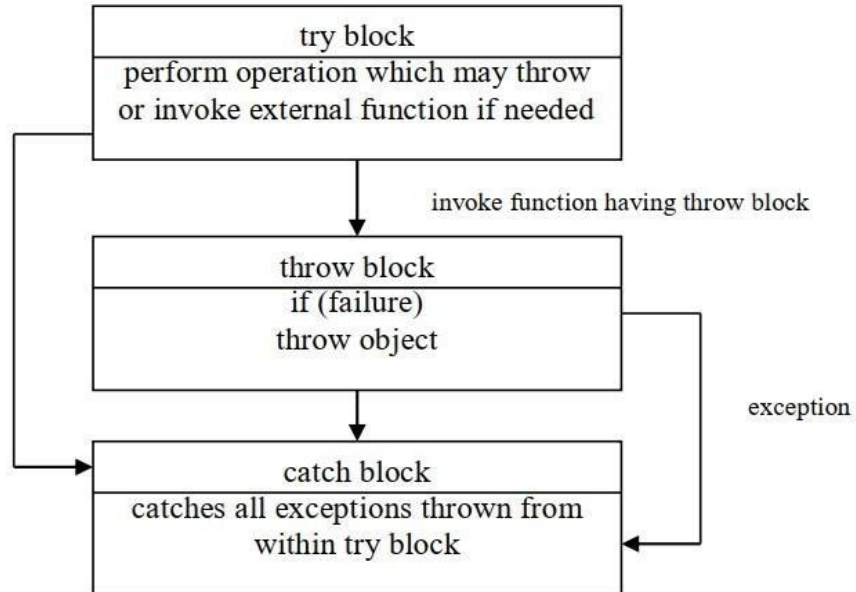


Figure 11.1 Exception Handling Flowchart

11.2 throw Construct

The keyword throw is used to raise an exception when an error is generated in the computation.

The throw expression initialize a temporary object of the type T used in throw (T arg).

syntax:

```
throw T;
```

11.3 catch Construct

The exception handler is indicated by the catch keyword. It must be used immediately after the statements marked by the try keyword. The catch handler can also occur immediately after another catch Each handler will only evaluate an exception that matches.

syn:

```
catch(T)
{
    // error messages
}
```

11.4 try construct

The try keyword defines a boundary within which an exception can occur. A block of code in which an exception can occur must be prefixed by the keyword try. Following the try keyword is a block of code enclosed by braces. This indicates that the prepared to test for the existence of exceptions. If an exception occurs, the program flow is interrupted.

```
try
{ ...
if (failure)
    throw T;
}
catch(T)
{ ...
```

MANAGING CONSOLE I/O

Chapter – 12

12.1 Introduction

One of the most essential features of interactive programming is its ability to interact with the users through operator console usually comprising keyboard and monitor. Accordingly, every computer language (and compiler) provides standard input/output functions and/or methods to facilitate console operations.

C++ accomplishes input/output operations using concept of stream. A stream is a series of bytes whose value depends on the variable in which it is stored. This way, C++ is able to treat all the input and output operations in a uniform manner. Thus, whether it is reading from a file or from the keyboard, for a C++ program it is simply a stream.

We have used the objects `cin` and `cout` (pre-defined in the `iostream.h` file) for the input and output of data of various types. This has been made possible by overloading the operators `>>` and `<<` to recognize all the basic C++ types. The `>>` operator is overloaded in the `istream` class and is overloaded in the `ostream` class. The following is the general format for reading data from the keyboard:

```
cin >> variable1 >> variable2 >>... ...>> variableN;
```

Where `variable1`, `variable2`,.....are valid C++ variable names that have been declared already. This statement will cause the computer to halt the execution and look for input data from the keyboard. The input data for this statement would be:

```
data1 data2.....dataN
```

The input data are separated by white spaces and should match the type of variable in the `cin` list. Spaces, newlines and tabs will be skipped.

The operator `>>` reads the data character by character and assigns it to the indicated location. The reading for a variable will be terminated at the encounter of a white space or a character that does not match the destination type.

For example, consider the following code:

```
int code;  
cin >> code;
```

Suppose the following data is given as input:

1267E

The operator will read the characters up to 7 and the value 1267 is assigned to code. The character E remains in the input stream and will be input to the next cin statement. The general format of outputting data:

```
cout << item1 << item2 <<.....<< itemN;
```

The items, item1 through itemN may be variables or constants of any basic types.

12.2 The put() and get() Functions

The classes istream and ostream define two member functions get() and put() respectively to handle the single character input/output operations. There are two types of get() functions. We can use both get(char*) and get(void) prototypes to fetch a character including the blank space, tab and the newline character. The get(char*) version assigns the input character to its argument and the get(void) version returns the input character.

Since these functions are members of the input/output stream classes, we must invoke them using an appropriate object. For instance, look at the code snippet given below:

```
char c;
cin.get (c); //get a character from keyboard and
assign it to c while (c!= '\n')
{
    cout << C; //display the character on screen cin.get (c);
    //get another character
}
```

This code reads and displays a line of text (terminated by a newline character).

Remember, the operator> >can also be used to read a character but it will skip the white spaces and newline character. The above while loop will not work properly if the statement

```
cin >> c;
```

is used in place of

```
cin.get (c);
```

Try using both of them and compare the results. The get(void) version is used as follows:

```
char c;
```

```
c = cin.get(); //cin.get (c) replaced
```

The value returned by the function `get()` is assigned to the variable `c`.

The function `put()`, a member of `ostream` class, can be used to output a line of text, character by character. For example,

```
cout << put ('x');
```

displays the character `x` and

```
cout << put (ch) ;
```

displays the value of variable `ch`.

The variable `ch` must contain a character value. We can also use a number as an argument to the function `put ()`. For example,

```
cout << put (68) ;
```

displays the character `D`. This statement will convert the `int` value `90` to a `char` value and display the character whose ASCII value is `68`,

12.3 The `getline ()` and `write ()` Functions

We can read and display a line of text more efficiently using the line-oriented input/output functions `getline()` and `write()`. The `getline()` function reads a whole line of text that ends with a newline character. This function can be invoked by using the object `cin` as follows:

```
cin.getline(line, size);
```

This function call invokes the function which reads character input into the variable `line`. The reading is terminated as soon as either the newline character `'\n'` is encountered or size number of characters are read (whichever occurs first). The newline character is read but not saved. Instead, it is replaced by the null character.

For example; consider the following code:

```
char name [20] ;
```

```
cin.getline(name, 20);
```

Assume that we have given the following input through the keyboard:

Neeraj good

This input will be read correctly and assigned to the character array `name`. Let us suppose the input is as follows:

```
Object Oriented Programming
```

In this case, the input will be terminated after reading the following 19 characters:

Object Oriented Pro

After reading the string/ cin automatically adds the terminating null character to the character array.

Remember, the two blank spaces contained in the string are also taken into account, i.e.

between Objects and Oriented and Pro. We can also read strings using the operator >> as follows:

```
cin >> name;
```

But remember cin can read strings that do not contain white space. This means that cin can read just one word and not a series of words such as “Neeraj good”.

12.4 Formatted Console I/O Operations

C++ supports a number of features that could be used for formatting the output. These features include:

1. ios class functions and flags.
2. Manipulators.
3. User-defined output functions.

The ios class contains a large number of member functions that could be used to format the output in a number of ways. The most important ones among them are listed below.

Function	Task
width()	To specify the required field size for displaying an output value
Precision()	To specify the number of digits to be displayed after the decimal point of a float value
fill()	To specify a character that is used to fill the unused portion of a field.
self()	To specify format flags that can control the form of output display (such as Left-justification and right-justification).
Unself()	To clear the flags specified.

Table 12.1: I/O Manipulators

Manipulators are special functions that can be included in these statements to alter the format parameters of a stream. The table given below shows some important! manipulator functions that are frequently used. To access these manipulators, the file iomanip.h should be included in the program.

Manipulator	Equivalent ios function
setw()	width()
Setprecision()	Precision()
Setfill()	fill()
setiosflags()	self()
Resetiosflags()	Unself()

Table 12.2 I/O Manipulators and Their Equivalent ios Function

In addition to these functions supported by the C++ library, we can create our own manipulator functions to provide any special output formats.

12.5 Streams

C++ is designed to work with a wide variety of devices including terminals, disks, and tape drives. Although each device is very different, the system supplies an interface to the programmer that is independent of the actual device being accessed, This interface is known as stream.

A stream is a sequence of bytes. It acts either as a source from which the input data can be obtained or as a destination to which the output data can be sent. The source stream that provides data to the program is called the input stream. In other words, a program extracts the bytes from an input stream and inserts bytes into an output stream.

The data in the input stream can come from the keyboard or any other storage device.

Similarly, the data in the output stream can go to the screen or any other storage device. As mentioned earlier, a stream acts as an interface between the program and the input/output device.

Therefore, a C++ program handles data (input or output) independent of the devices used. C++ contains several pre-defined streams that are automatically opened when a program begins its execution. They include cin and cout which have been used very often in our earlier programs. We know that cin represents the input stream connected to the standard input device (usually the keyboard) and cout represents the output stream connected to the standard output device (usually the screen). Note that the keyboard and the screen are default options. We can redirect streams to other devices or files, if necessary.

12.6 I/O Operations

Input and Output statements of computer languages are used to provide communications between the user and the program. In most of the computer languages, input and output are done through statements. But in C++, these operations are carried out through its built-in functions. The I/O functions are designed in header files like `fstream.h`, `iostream.h` etc.

Through these functions, data can be read from or written to files or standard input/output devices like keyboard and VDU. This execution of a program can be interrupted by input/output calls. Hence the data can be entered or output can be retrieved during execution. The file, stream classes support a number of member functions for performing the input and output operations on files. One pair of functions, `put( )` and `get( )`, are designed for handling a single character at a time. Another pair of functions, `write()` and `readQ`, are designed to write and read blocks of binary data.

12.7 `put()` and `get()` function

The function `put( )` writes a single character to the associated stream. Similarly, the function `get( )` reads a single character from the, associated stream. The program, requests for a string. On receiving the string, the program writes it, character, by character, to the file using the `put()` function in a for loop. Note that the length of the string is used to terminate the for loop. C++ provides a number of useful predefined stream classes for console input/output operations. Some of the C++ the predefined stream objects are listed below.

`cin` This is the name of standard input stream, usually keyboard. The corresponding name in C is `stdin`.

`cout` This is the name of standard output stream, usually screen of the monitor. The corresponding name in C is `stdout`.

`cerr` This is the name of standard error output stream, usually screen of the monitor. The corresponding name in C is `stderr`.

`clog` This is another version of `cerr`. It provides buffer to collect errors. C does not have a stream equivalent to this.

In their default roles, these streams are tied up with the keyboard and screen of the monitor as describe above. However, you can redirect them from and to other devices and files.

12.7 Keywords

put (): A member of ostream class, could be used to output a line of text, character by character.

Get (): A member of istream class, used to input a single character at a line.

Getline (): The get line () function reads a whole line of -text that ends with a new line character. This function could be invoked by using the object cin.

Manipulators: Special functions that can be included in console I/O statements to alter the format-parameters of a stream

12.7.1 Streams:

C++ is designed to work with a wide variety of devices including, disks and tape drives. Although each device is very different the system supplies an interface to the programmer that is independent of the actual device accessed. This interface is known as stream.

12.7.2 Output Streams:

The source stream that provides data to the program is called the. output stream

NAMESPACES

13.1 Scopes

Named entities, such as variables, functions, and compound types need to be declared before being used in C++. The point in the program where this declaration happens influences its visibility:

An entity declared outside any block has global scope, meaning that its name is valid anywhere in the code. While an entity declared within a block, such as a function or a selective statement, has block scope, and is only visible within the specific block in which it is declared, but not outside it.

Variables with block scope are known as local variables.

For example, a variable declared in the body of a function is a local variable that extends until the end of the the function (i.e., until the brace } that closes the function definition)

The visibility of an entity with block scope extends until the end of the block, including inner blocks. Nevertheless, an inner block, because it is a different block, can re-utilize a name existing in an outer scope to refer to a different entity; in this case, the name will refer to a different entity only within the inner block, hiding the entity it names outside. While outside it, it will still refer to the original entity.

13.2 Namespaces

Only one entity can exist with a particular name in a particular scope. This is seldom a problem for local names, since blocks tend to be relatively short, and names have particular purposes within them, such as naming a counter variable, an argument, etc...

But non-local names bring more possibilities for name collision, especially considering that libraries may declare many functions, types, and variables, neither of them local in nature, and some of them very generic.

Namespaces allow us to group named entities that otherwise would have global scope into narrower scopes, giving them namespace scope. This allows organizing the elements of programs into different logical scopes referred to by names.

Conclusion

Completing this C++ training has been an important step in my learning journey, and it has helped me build a strong and clear understanding of programming. Through this training, I became familiar with the fundamental ideas of C++, such as variables, data types, operators, loops, arrays, functions, and object-oriented programming. Learning these topics has shown me how powerful and flexible C++ can be when solving different kinds of problems. Even though some concepts were challenging at first, consistent practice made them easier to understand, and I now feel more confident in applying them.

This training also improved the way I think and approach problems. C++ requires careful planning, logical thinking, and attention to detail, and working with the language helped me develop these skills. I learned how important it is to write clear code, keep things organized, and break problems into smaller parts. These habits not only help in C++ but also make programming in general much easier.

Another important part of the training was understanding how different features of the language work together. I learned how functions make code reusable, how loops help repeat tasks, how arrays store multiple values, and how classes and objects help create structured programs. Understanding these relationships has helped me see programming in a more systematic way.

Overall, this C++ training has strengthened my foundation in programming and prepared me for more advanced learning. I now have the confidence to explore deeper concepts like file handling, pointers, templates, and other advanced features of the language. I am also better prepared to learn other programming languages because many of the skills I learned in C++ apply to programming in general. This training has motivated me to keep improving and continue my journey in the world of programming.

References

- [1] <https://www.w3schools.com>
- [2] <https://www.geeksforgeeks.org>
- [3] <https://www.tutorialspoint.com>
- [4] <https://www.freecodecamp.org>
- [5] <https://www.learncpp.com>